La ricerca e il problem solving

LECTURE/"02

Andrea Augello

Department of Engineering, University of Palermo, Italy

Intelligenza Artificiale 1

Laurea Magistrale in Ingegneria Informatica



Università degli Studi di Palermo

Obiettivi

Alla fine di questa esercitazione sarete in grado di:

- Implementare un agente di ricerca in ampiezza generico basato su una frontiera di cammini.
- Dato un problema con stati, azioni, vincoli e obiettivo, formalizzarlo come ricerca su grafo.
- Scegliere una rappresentazione dello stato e motivarla rispetto a completezza, validità ed efficienza.
- Disinnescare una bomba.

Ricerca su grafi

Ricerca su grafi

Molti problemi possono essere modellati come dei problemi di ricerca su grafi.

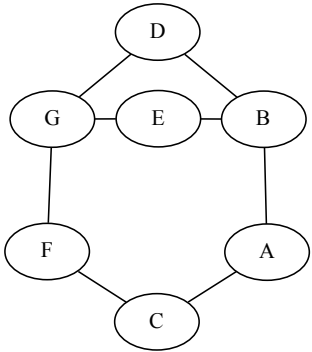
- **Stato iniziale**: stato da cui parte la ricerca
- **Stato finale**: stato che si vuole raggiungere
- **Stato corrente**: stato in cui si trova l'algoritmo
- **Operatore**: azione che modifica lo stato corrente
- **Cammino**: sequenza di stati che porta dallo stato iniziale a quello corrente
- **Costo del cammino**: costo totale per raggiungere lo stato corrente

Non tutte queste informazioni sono necessarie e/o disponibili per ogni problema.

Ricerca su grafi

- Iniziamo modellando un grafo in modo esplicito.
- Il grafo è rappresentabile con un dizionario di liste (di adiacenza).
- Le chiavi del dizionario esterno sono i nodi del grafo.
- Se il grafo non è orientato ogni arco è rappresentato due volte.

```
graph = {'A': ['B', 'C'],
         'B': ['A', 'D', 'E'],
         'C': ['A', 'F'],
         'D': ['B', 'G'],
         'E': ['B', 'G'],
         'F': ['C', 'G'],
         'G': ['D', 'E', 'F']}
```



In questo primo esempio, il problema è quello di trovare un cammino che porti dal nodo `A` al nodo `D` (se esiste) attraversando il minor numero di archi.

- **Stato iniziale**: `A`
- **Stato finale**: `D`
- **Stato corrente**: Il nodo in cui si trova l'algoritmo ad ogni passo
- **Operatore**: Spostarsi su un nodo adiacente
- **Costo del cammino**: Numero di archi attraversati

Volendo attraversare il minor numero di archi, possiamo utilizzare una ricerca in ampiezza.

Ricerca su grafi

Implementiamo un agente che esegue una ricerca in ampiezza.

```
class Agent():
    def __init__(self, graph, start, goal):
        self.graph = graph
        self.goal  = goal
        self.frontier = [[start]]

    def next_states(self, path):
        pass

    def is_goal(self, state):
        pass

    def bfs(self):
        pass
```

L’algoritmo BFS

1. Se la frontiera è vuota, restituisci `None`
2. Prendi il primo cammino dalla frontiera
3. Se l’ultimo stato del cammino è uno stato goal, restituisci il cammino
4. Altrimenti, genera tutti i cammini ottenibili aggiungendo un nuovo stato alla fine del cammino (evitando cicli)
5. Aggiungi i nuovi cammini alla frontiera
6. Ripeti dal passo 1

Ricerca su grafi

Implementazioni alternative:

- Riusciamo ad implementare lo stesso comportamento rappresentando il grafo come lista di archi?
- La ricerca in ampiezza può essere implementata in modo ricorsivo?

Ricerca su grafi

Non sempre è possibile modellare un grafo in modo esplicito.

- Il numero di stati può essere infinito.
- Il grafo può essere troppo grande per essere memorizzato.
- Il grafo può essere dinamico.
- L'adiacenza può essere determinata da una funzione.

La funzione `next_states` può quindi dover fare valutazioni meno banali.

- Facciamo quindi uso di operatori di trasformazione di stato
- Dato uno stato ed una azione, un operatore di trasformazione di stato restituisce un nuovo stato.

Risolvere problemi reali

Un problema di ricerca non è necessariamente un grafo esplicito, ma può essere modellato come tale tramite:

- stati
- azioni/operatori
- stato iniziale
- goal
- costo del cammino

Il grafo può essere implicito.
Una volta modellato il problema, definendo transizioni tra stati e vincoli sugli stati, possiamo utilizzare lo stesso algoritmo di ricerca in ampiezza per risolvere più problemi diversi.

Disinnescare una bomba

Disinnescare una bomba

Die Hard with a Vengeance (1995)

https://youtu.be/2vdF6NASMiE?t=45

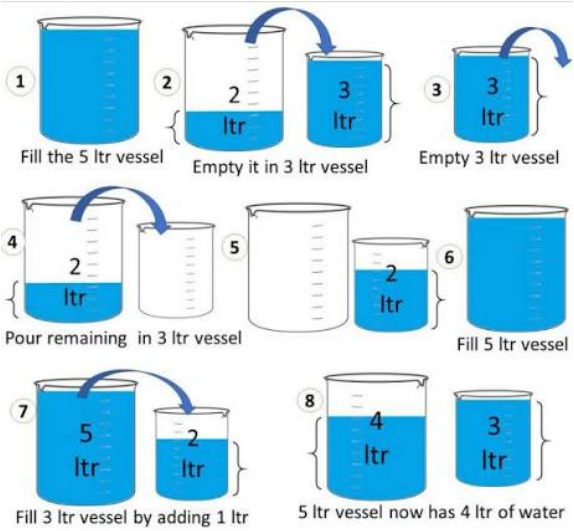
Il problema dei due secchi

Sia dato il seguente problema: si hanno a disposizione due secchi di capacità rispettivamente 3 e 5 litri, inizialmente vuoti, e un grosso contenitore da 20 litri pieno d'acqua. L'obiettivo è quello di ottenere esattamente 4 litri nel secchio da 5 litri.

Le azioni possibili sono:

- riempire un secchio dal contenitore;
- svuotare un secchio nel contenitore;
- travasare da un secchio all'altro finché uno non è pieno o l'altro non è vuoto.

Il problema dei due secchi



Come modelliamo uno stato?

Ci serve sapere quanta acqua c’è in ogni secchio,

Come modelliamo uno stato?

Ci serve sapere quanta acqua c'è in ogni secchio,

Possiamo rappresentare uno stato come una tripla di interi: (quantità nel secchio da 3 litri, quantità nel secchio da 5 litri, quantità nel contenitore da 20 litri).

Come modelliamo uno stato?

Ci serve sapere quanta acqua c'è in ogni secchio,

Possiamo rappresentare uno stato come una tripla di interi: (quantità nel secchio da 3 litri, quantità nel secchio da 5 litri, quantità nel contenitore da 20 litri).

Possiamo rappresentare uno stato come un dizionario con chiave la capacità del secchio e valore la quantità di acqua in esso contenuta.

Come modelliamo uno stato?

Ci serve sapere quanta acqua c'è in ogni secchio,

Possiamo rappresentare uno stato come una tripla di interi: (quantità nel secchio da 3 litri, quantità nel secchio da 5 litri, quantità nel contenitore da 20 litri).

Possiamo rappresentare uno stato come un dizionario con chiave la capacità del secchio e valore la quantità di acqua in esso contenuta.

Quale rappresentazione è più efficiente? Quale è più generalizzabile?

Inizializziamo l'agente

```
class Agent:
    def __init__(self, jugs=(3, 5, 20), start= (0, 0, 20), target=4):
        self.jugs = jugs
        self.target = target
        self.frontier = [[start]]
```

Il costruttore dell'agente prende in input un po' di informazioni sul problema, come la capacità dei secchi, la quantità iniziale di acqua nel contenitore, e la quantità di acqua che vogliamo ottenere in uno dei secchi.

Inizializziamo l'agente

```
class Agent:
    def __init__(self, jugs=(3, 5, 20), start= (0, 0, 20), target=4):
        self.jugs = jugs
        self.target = target
        self.frontier = [[start]]
```

Il costruttore dell'agente prende in input un po' di informazioni sul problema, come la capacità dei secchi, la quantità iniziale di acqua nel contenitore, e la quantità di acqua che vogliamo ottenere in uno dei secchi.

Scrivere un programma che risolva una singola istanza del problema è relativamente semplice. Vista la natura generica degli algoritmi di ricerca, è più interessante (e valutato con punteggi maggiori all'esame) scrivere soluzioni il più generali possibile, in modo da poterle riutilizzare per risolvere problemi simili.

Possiamo avere un grafo degli stati esplicito?

Il numero di stati possibili è finito, quindi potremmo costruire un grafo esplicito degli stati e delle transizioni tra di essi.

Possiamo avere un grafo degli stati esplicito?

Il numero di stati possibili è finito, quindi potremmo costruire un grafo esplicito degli stati e delle transizioni tra di essi.

Generare tutti questi possibili stati però è un pessimo modo per passare il nostro tempo. Nella nostra istanza del problema, ci sono $4 \times 6 \times 21 = 504$ stati possibili, ma molti di essi non sono validi (e.g., somma totale dei litri diversa da 20) o non sono raggiungibili.

Il conto alla rovescia della bomba intanto va avanti…

Come modelliamo un’azione?

Le azioni possibili sono:

- riempire un secchio dal contenitore;
- svuotare un secchio nel contenitore;
- travasare da un secchio all’altro finché uno non è pieno o l’altro non è vuoto.

Come modelliamo un'azione?

Le azioni possibili sono:

- riempire un secchio dal contenitore;
- svuotare un secchio nel contenitore;
- travasare da un secchio all'altro finché uno non è pieno o l'altro non è vuoto.

La nostra azione è univocamente determinata da sorgente e destinazione, quindi possiamo rappresentarla come una tupla `(src, dst)`.

Come generiamo i next states?

Per generare i next states, dobbiamo considerare tutte le azioni possibili, e applicarle allo stato corrente.

- Data una sorgente e una destinazione, possiamo versare dalla sorgente alla destinazione finché la sorgente non è vuota o la destinazione non è piena.

Next states

```
    def next_states(self, state):
        def is_valid(state):
            return all(0 <= curr <= cap for curr, cap in zip(state,
                self.jugs))

        moves = ((i,j) for i in range(len(self.jugs)) for j in
            range(len(self.jugs)) if i != j)
        for src, dst in moves:
            amount = min(state[src], self.jugs[dst] - state[dst])
            new_state = list(state)
            new_state[src] -= amount
            new_state[dst] += amount
            if is_valid(new_state):
                yield tuple(new_state)
```

Il controllo di `is_valid` non è strettamente necessario dato il modo in cui generiamo i next states, ma non sempre è così semplice generare next states validi, quindi è buona norma avere un controllo di validità.

Il resto è riciclo

```
    def is_goal(self, path): #potrebbe stare inline nella BFS
        return self.target in path[-1]

    def bfs(self):
        while self.frontier:
            path = self.frontier.pop(0)
            if self.is_goal(path):
                return path
            for s in self.next_states(path[-1]):
                if s not in path:
                    self.frontier.append(path + [s])

if __name__ == "__main__":
    agent = Agent()
    print(agent.bfs())
```

```
>> [(0, 0, 20), (0, 5, 15), (3, 2, 15), (0, 2, 18), (2, 0, 18), (2, 5, 13), (3, 4, 13)]
```

Cannibali e missionari

Cannibali e missionari

Tre missionari e tre cannibali devono attraversare un fiume.

- Missionari e cannibali sono inizialmente sulla sponda sinistra del fiume e devono raggiungere la sponda destra.
- Per farlo, hanno a disposizione una barca che può contenere al massimo due persone.
- La barca non può attraversare il fiume senza nessuno a bordo.
- Il numero di cannibali non può mai essere superiore al numero di missionari su una delle due sponde.

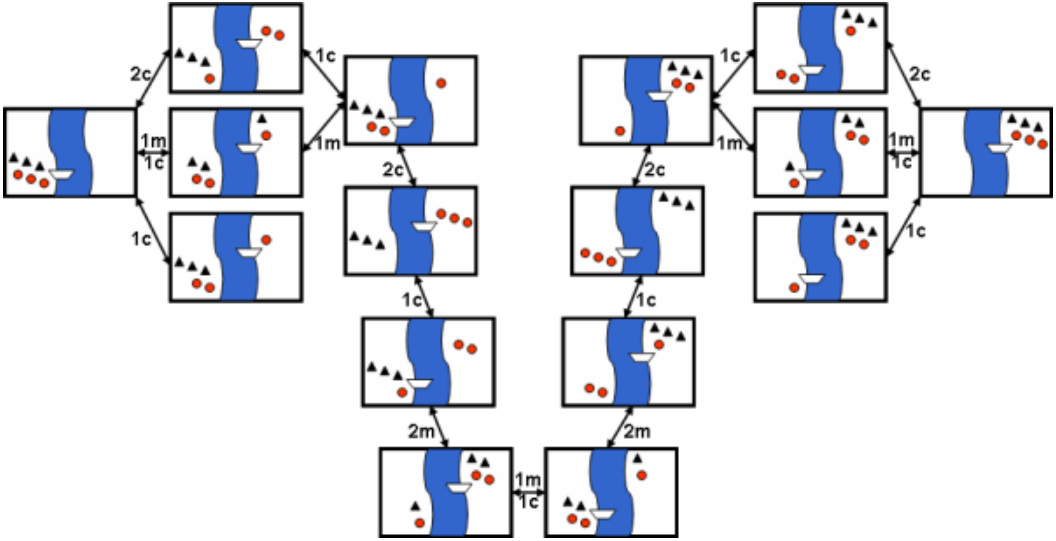
L'obbiettivo è quello di trovare una sequenza minima di azioni che permetta a tutti di attraversare il fiume.

Cannibali e missionari - Proviamo a risolverlo

Se dovessimo risolvere questo problema a mano, come procederemmo?

- Inizialmente, possiamo solo portare due cannibali o due missionari o un cannibale e un missionario.
- Se portiamo due missionari, sulla sponda sinistra rimarrebbero 3 cannibali e 1 missionario, e i cannibali mangerebbero il missionario.
- Se portiamo una sola persona, al ritorno dovrebbe tornare indietro.
- Chi portiamo a destra?
- …

Cannibali e missionari



Cannibali e missionari

Guardiamo lo scheletro di un agente che risolve il problema.

```
class Agent():
    def __init__(self):
        pass

    @staticmethod
    def apply_move(state, move):
        pass # Applica una mossa ad uno stato

    def next_states(self, state):
        pass # Genera tutti gli stati raggungibili da uno stato

    def is_goal(self, state):
        pass # Verifica se uno stato sia lo stato finale

    def bfs(self):
        pass # Implementa la ricerca in ampiezza
```

Rappresentazione dello stato

Il primo problema che bisogna porsi in un problema di ricerca è come rappresentare lo stato.

Rappresentazione dello stato

Il primo problema che bisogna porsi in un problema di ricerca è come rappresentare lo stato.

Spesso questo è il passo più difficile che va a determinare quando sarà complesso/efficiente implementare l'algoritmo di ricerca.

Rappresentazione dello stato

Il primo problema che bisogna porsi in un problema di ricerca è come rappresentare lo stato.

Spesso questo è il passo più difficile che va a determinare quando sarà complesso/efficiente implementare l'algoritmo di ricerca.

Come scegliere in che modo rappresentare lo stato?

- Bisogna assicurarsi che lo stato contenga tutte le informazioni necessarie per verificare i vincoli del problema.
- I dettagli irrilevanti per il problema non devono essere memorizzati nello stato (e.g., i missionari sono fungibili tra loro).
- La rappresentazione deve (idealmente) essere efficiente da manipolare (se controllare la validità di uno stato è costoso, l'algoritmo di ricerca sarà lento).

Rappresentazione dello stato

In questo caso, scegliamo di rappresentare lo stato come un dizionario:

```
start = {'L': {'m': 3, 'c': 3},
         'R': {'m': 0, 'c': 0},
         'boat':  'L'}
```

```
goal = {'L': {'m': 0, 'c': 0},
        'R': {'m': 3, 'c': 3},
        'boat':  'R'}
```

Rappresentazione dello stato

Quali altri opzioni avremmo avuto per rappresentare lo stato?

Rappresentazione dello stato

Quali altri opzioni avremmo avuto per rappresentare lo stato?

- Una tupla di 5 elementi: numero di missionari a sinistra, numero di cannibali a sinistra, numero di missionari a destra, numero di cannibali a destra, posizione della barca (Codice difficile da leggere).
- Invertire il dizionario: per ogni categoria di persona, memorizzare quante sono a sinistra e quante a destra (Controlli e manipolazioni più complesse).
- Tenere traccia solo di una sponda e calcolare l'altra (Più conti, meno spazio occupato).

Movimenti possibili

Rappresentare esplicitamente tutti gli stati possibili e le loro transizioni sarebbe troppo dispendioso.

Definiamo quindi una lista di azioni possibili che, applicate ad uno stato, generano un nuovo stato.

Movimenti possibili

Rappresentare esplicitamente tutti gli stati possibili e le loro transizioni sarebbe troppo dispendioso.

Definiamo quindi una lista di azioni possibili che, applicate ad uno stato, generano un nuovo stato.

Data la posizione della barca, le azioni possibili sono:

- `{'m': 1, 'c': 0}`: spostare un missionario
- `{'m': 0, 'c': 1}`: spostare un cannibale
- `{'m': 1, 'c': 1}`: spostare un missionario e un cannibale
- `{'m': 2, 'c': 0}`: spostare due missionari
- `{'m': 0, 'c': 2}`: spostare due cannibali

Non tutte le azioni sono però sempre possibili o risultano in uno stato valido.

Controllare la validità di uno stato

Necessitiamo di una funzione che ci permetta di verificare se uno stato è valido secondo le regole del problema per rimuovere gli stati non validi dalla frontiera.

Uno stato non è valido se su una delle due sponde sono presenti missionari in minoranza rispetto ai cannibali (o se il numero di missionari o cannibali è negativo).

Controllare la validità di uno stato

Necessitiamo di una funzione che ci permetta di verificare se uno stato è valido secondo le regole del problema per rimuovere gli stati non validi dalla frontiera.

Uno stato non è valido se su una delle due sponde sono presenti missionari in minoranza rispetto ai cannibali (o se il numero di missionari o cannibali è negativo).

```
        def is_valid(state):
            for side in ['L', 'R']:
                if state[side]['m'] < 0 or state[side]['c'] < 0:
                    return False
                if state[side]['m'] > 0 and state[side]['c'] >
                    state[side]['m']:
                    return False
            return True
```

Applicare una mossa

Dato uno stato e una mossa, dobbiamo generare un nuovo stato. Per prima cosa copiamo lo stato corrente. Cambiamo la posizione della barca, e rimuoviamo le persone dalla sponda di partenza e le aggiungiamo alla sponda di arrivo.

Questo metodo è definito come statico perché non ha bisogno di accedere a nessuna variabile della classe.

Applicare una mossa

Dato uno stato e una mossa, dobbiamo generare un nuovo stato. Per prima cosa copiamo lo stato corrente. Cambiamo la posizione della barca, e rimuoviamo le persone dalla sponda di partenza e le aggiungiamo alla sponda di arrivo.

Questo metodo è definito come statico perché non ha bisogno di accedere a nessuna variabile della classe.

```
    @staticmethod
    def apply_move(state, move):
        new_state = copy.deepcopy(state)
        new_state['boat'] = 'L' if state['boat'] == 'R' else 'R'
        for person in ['m', 'c']:
            new_state[state['boat']][person] -= move[person]
            new_state[new_state['boat']][person] += move[person]
        return new_state
```

Applicare una mossa

Dato uno stato e una mossa, dobbiamo generare un nuovo stato. Per prima cosa copiamo lo stato corrente. Cambiamo la posizione della barca, e rimuoviamo le persone dalla sponda di partenza e le aggiungiamo alla sponda di arrivo.

Questo metodo è definito come statico perché non ha bisogno di accedere a nessuna variabile della classe.

```
    @staticmethod
    def apply_move(state, move):
        new_state = copy.deepcopy(state)
        new_state['boat'] = 'L' if state['boat'] == 'R' else 'R'
        for person in ['m', 'c']:
            new_state[state['boat']][person] -= move[person]
            new_state[new_state['boat']][person] += move[person]
        return new_state
```

A che serve la funzione `copy.deepcopy`?

Generazione degli stati successivi

Creiamo nuovi stati prendendo in considerazione tutte le azioni possibili.
Solamente le azioni che non violano le regole del problema verranno restituite.

Generazione degli stati successivi

Creiamo nuovi stati prendendo in considerazione tutte le azioni possibili.
Solamente le azioni che non violano le regole del problema verranno restituite.

```
    def next_states(self, state):
        def is_valid(state):
            for side in ['L', 'R']:
                if state[side]['m'] < 0 or state[side]['c'] < 0:
                    return False
                if state[side]['m'] > 0 and state[side]['c'] >
                    state[side]['m']:
                    return False
            return True

        for move in self.moves:
            new_state = self.apply_move(state, move)
            if is_valid(new_state):
                yield new_state
```

Generazione degli stati successivi

Creiamo nuovi stati prendendo in considerazione tutte le azioni possibili.
Solamente le azioni che non violano le regole del problema verranno restituite.

```
    def next_states(self, state):
        def is_valid(state):
            for side in ['L', 'R']:
                if state[side]['m'] < 0 or state[side]['c'] < 0:
                    return False
                if state[side]['m'] > 0 and state[side]['c'] >
                    state[side]['m']:
                    return False
            return True

        for move in self.moves:
            new_state = self.apply_move(state, move)
            if is_valid(new_state):
                yield new_state
```

Implementazione alternativa:

```
return (new_state for move in self.moves if is_valid(new_state :=
    self.apply_move(state, move)))
```

La ricerca in ampiezza

A questo punto, possiamo eseguire una ricerca in ampiezza per trovare una sequenza di stati che porti dallo stato iniziale a quello finale.

Il livello di astrazione è sufficientemente alto da poter riutilizzare il codice della ricerca in ampiezza che abbiamo visto in precedenza.

La ricerca in ampiezza

A questo punto, possiamo eseguire una ricerca in ampiezza per trovare una sequenza di stati che porti dallo stato iniziale a quello finale.

Il livello di astrazione è sufficientemente alto da poter riutilizzare il codice della ricerca in ampiezza che abbiamo visto in precedenza.

Gli algoritmi di ricerca sono algoritmi generici che possono essere riutilizzati in molti problemi diversi una volta che si è in grado di modellare il problema col giusto livello di astrazione.

Sfida per casa

Sfida per casa

Provate a scrivere un agente analogo per il rompicapo di capra, lupo e cavolo:

- Un contadino deve attraversare un fiume con una barca che può trasportare solo lui e (opzionalmente) uno tra una capra, un lupo o un cavolo.
- Se il lupo e la capra rimangono soli insieme, il lupo mangerà la capra.
- Se la capra e il cavolo rimangono soli insieme, la capra mangerà il cavolo.
- Come può fare il contadino a portate tutti e tre gli oggetti sull'altra sponda del fiume?

Feedback



https://app.wooclap.com/NRGKEGO/questionnaires/6ab374dc97cc64c5bd46c587